

Develop with Redis

What's new?

Quick starts

Client tools

Client APIs

Understand data types

Interact with data

Redis for AI

Use Redis

Reference

- Command tips

- Client-side caching

- Modules API

- Sentinel clients

- Command key specifications

- Protocol spec

- Client handling

- Eviction

- Command arguments

- Gopher protocol

Libraries and tools

Redis products

Commands

Redis serialization protocol specification

Redis serialization protocol (RESP) is the wire protocol that clients implement

To communicate with the Redis server, Redis clients use a protocol called Redis Serialization Protocol (RESP). While the protocol was designed specifically for Redis, you can use it for other client-server software projects.

RESP is a compromise among the following considerations:

- Simple to implement.
- Fast to parse.
- Human readable.

RESP can serialize different data types including integers, strings, and arrays. It also features an error-specific type. A client sends a request to the Redis server as an array of strings. The array's contents are the command and its arguments that the server should execute. The server's reply type is command-specific.

RESP is binary-safe and uses prefixed length to transfer bulk data so it does not require processing bulk data transferred from one process to another.

RESP is the protocol you should implement in your Redis client.



Note:

The protocol outlined here is used only for client-server communication. [Redis Cluster](#) uses a different binary protocol for exchanging messages between nodes.

RESP versions

Support for the first version of the RESP protocol was introduced in Redis 1.2. Using RESP with Redis 1.2 was optional and had mainly served the purpose of working the kinks out of the protocol.

In Redis 2.0, the protocol's next version, a.k.a RESP2, became the standard communication method for clients with the Redis server.

RESP3 is a superset of RESP2 that mainly aims to make a client author's life a little bit easier. Redis 6.0 introduced experimental opt-in support of RESP3's features (excluding streaming strings and streaming aggregates). In addition, the introduction of the HELLO command allows clients to handshake and upgrade the connection's protocol version (see Client handshake).

Up to and including Redis 7, both RESP2 and RESP3 clients can invoke all core commands. However, commands may return differently typed replies for different protocol versions.

Future versions of Redis may change the default protocol version, but it is unlikely that RESP2 will become entirely deprecated. It is possible, however, that new features in upcoming versions will require the use of RESP3.

Network layer

A client connects to a Redis server by creating a TCP connection to its port (the default is 6379).

While RESP is technically non-TCP specific, the protocol is used exclusively with TCP connections (or equivalent stream-oriented connections like Unix sockets) in the context of Redis.

Request-Response model

The Redis server accepts commands composed of different arguments. Then, the server processes the command and sends the reply back to the client.

This is the simplest model possible; however, there are some exceptions:

- Redis requests can be pipelined. Pipelining enables clients to send multiple commands at once and wait for replies later.
- When a RESP2 connection subscribes to a Pub/Sub channel, the protocol changes semantics and becomes a *push* protocol. The client no longer requires sending commands because the server will automatically send new messages to the client (for the channels the client is subscribed to) as soon as they are received.
- The MONITOR command. Invoking the MONITOR command switches the connection to an ad-hoc push mode. The protocol of this mode is not specified but is obvious to parse.

- Protected mode. Connections opened from a non-loopback address to a Redis while in protected mode are denied and terminated by the server. Before terminating the connection, Redis unconditionally sends a -DENIED reply, regardless of whether the client writes to the socket.
- The RESP3 Push type. As the name suggests, a push type allows the server to send out-of-band data to the connection. The server may push data at any time, and the data isn't necessarily related to specific commands executed by the client.

Excluding these exceptions, the Redis protocol is a simple request-response protocol.

RESP protocol description

RESP is essentially a serialization protocol that supports several data types. In RESP, the first byte of data determines its type.

Redis generally uses RESP as a request-response protocol in the following way:

- Clients send commands to a Redis server as an array of bulk strings. The first (and sometimes also the second) bulk string in the array is the command's name. Subsequent elements of the array are the arguments for the command.
- The server replies with a RESP type. The reply's type is determined by the command's implementation and possibly by the client's protocol version.

RESP is a binary protocol that uses control sequences encoded in standard ASCII. The A character, for example, is encoded with the binary byte of value 65. Similarly, the characters CR (\r), LF (\n) and SP () have binary byte values of 13, 10 and 32, respectively.

The \r\n (CRLF) is the protocol's *terminator*, which **always** separates its parts.

The first byte in an RESP-serialized payload always identifies its type. Subsequent bytes constitute the type's contents.

We categorize every RESP data type as either *simple*, *bulk* or *aggregate*.

Simple types are similar to scalars in programming languages that represent plain literal values. Booleans and Integers are such examples.

RESP strings are either *simple* or *bulk*. Simple strings never contain carriage return (\r) or line feed (\n) characters. Bulk strings can contain any binary data and may also be referred to as *binary* or *blob*. Note that bulk strings may be further encoded and decoded, e.g. with a wide multi-byte encoding, by the client.

Aggregates, such as Arrays and Maps, can have varying numbers of sub-elements and nesting levels.

The following table summarizes the RESP data types that Redis supports:

RESP data type	Minimal protocol version	Category	First byte
<u>Simple strings</u>	RESP2	Simple	+
<u>Simple Errors</u>	RESP2	Simple	-
<u>Integers</u>	RESP2	Simple	:
<u>Bulk strings</u>	RESP2	Aggregate	\$
<u>Arrays</u>	RESP2	Aggregate	*
<u>Nulls</u>	RESP3	Simple	_
<u>Booleans</u>	RESP3	Simple	#
<u>Doubles</u>	RESP3	Simple	,
<u>Big numbers</u>	RESP3	Simple	(
<u>Bulk errors</u>	RESP3	Aggregate	!
<u>Verbatim strings</u>	RESP3	Aggregate	=
<u>Maps</u>	RESP3	Aggregate	%
<u>Attributes</u>	RESP3	Aggregate	`
<u>Sets</u>	RESP3	Aggregate	~
<u>Pushes</u>	RESP3	Aggregate	>

Simple strings

Simple strings are encoded as a plus (+) character, followed by a string. The string mustn't contain a CR (\r) or LF (\n) character and is terminated by CRLF (i.e., \r\n).

Simple strings transmit short, non-binary strings with minimal overhead. For example, many Redis commands reply with just "OK" on success. The encoding of this Simple String is the following 5 bytes:

```
+OK\r\n
```

When Redis replies with a simple string, a client library should return to the caller a string value composed of the first character after the + up to the end of the string, excluding the final CRLF bytes.

To send binary strings, use bulk strings instead.

Simple errors

RESP has specific data types for errors. Simple errors, or simply just errors, are similar to simple strings, but their first character is the minus (-) character. The difference between simple strings and errors in RESP is that clients should treat errors as exceptions, whereas the string encoded in the error type is the error message itself.

The basic format is:

```
-Error message\r\n
```

Redis replies with an error only when something goes wrong, for example, when you try to operate against the wrong data type, or when the command does not exist. The client should raise an exception when it receives an Error reply.

The following are examples of error replies:

```
-ERR unknown command 'asdf'  
-WRONGTYPE Operation against a key holding the wrong kind of value
```

The first upper-case word after the -, up to the first space or newline, represents the kind of error returned. This word is called an *error prefix*. Note that the error prefix is a convention used by Redis rather than part of the RESP error type.

For example, in Redis, ERR is a generic error, whereas WRONGTYPE is a more specific error that implies that the client attempted an operation against the wrong data type. The error prefix allows the client to understand the type of error returned by the server without checking the exact error message.

A client implementation can return different types of exceptions for various errors, or provide a generic way for trapping errors by directly providing the error name to the caller as a string.

However, such a feature should not be considered vital as it is rarely useful. Also, simpler client implementations can return a generic error value, such as `false`.

Integers

This type is a CRLF-terminated string that represents a signed, base-10, 64-bit integer.

RESP encodes integers in the following way:

```
: [<+|->]<value>\r\n
```

- The colon (:) as the first byte.
- An optional plus (+) or minus (-) as the sign.
- One or more decimal digits (0..9) as the integer's unsigned, base-10 value.
- The CRLF terminator.

For example, `:0\r\n` and `:1000\r\n` are integer replies (of zero and one thousand, respectively).

Many Redis commands return RESP integers, including INCR, LLEN, and LASTSAVE. An integer, by itself, has no special meaning other than in the context of the command that returned it. For example, it is an incremental number for INCR, a UNIX timestamp for LASTSAVE, and so forth. However, the returned integer is guaranteed to be in the range of a signed 64-bit integer.

In some cases, integers can represent true and false Boolean values. For instance, SISMEMBER returns 1 for true and 0 for false.

Other commands, including SADD, SREM, and SETNX, return 1 when the data changes and 0 otherwise.

Bulk strings

A bulk string represents a single binary string. The string can be of any size, but by default, Redis limits it to 512 MB (see the `proto-max-bulk-len` configuration directive).

RESP encodes bulk strings in the following way:

```
$<length>\r\n<data>\r\n
```

- The dollar sign (\$) as the first byte.
- One or more decimal digits (0..9) as the string's length, in bytes, as an unsigned, base-10 value.
- The CRLF terminator.
- The data.
- A final CRLF.

So the string "hello" is encoded as follows:

```
$5\r\nhello\r\n
```

The empty string's encoding is:

```
$0\r\n\r\n
```

Null bulk strings

Whereas RESP3 has a dedicated data type for null values, RESP2 has no such type. Instead, due to historical reasons, the representation of null values in RESP2 is via predetermined forms of the bulk strings and arrays types.

The null bulk string represents a non-existing value. The GET command returns the Null Bulk String when the target key doesn't exist.

It is encoded as a bulk string with the length of negative one (-1), like so:

```
$-1\r\n
```

A Redis client should return a nil object when the server replies with a null bulk string rather than the empty string. For example, a Ruby library should return `nil` while a C library should return `NULL` (or set a special flag in the reply object).

Arrays

Clients send commands to the Redis server as RESP arrays. Similarly, some Redis commands that return collections of elements use arrays as their replies. An example is the LRange command that returns elements of a list.

RESP Arrays' encoding uses the following format:

```
*<number-of-elements>\r\n<element-1>...<element-n>
```

- An asterisk (*) as the first byte.
- One or more decimal digits (0..9) as the number of elements in the array as an unsigned, base-10 value.
- The CRLF terminator.
- An additional RESP type for every element of the array.

So an empty Array is just the following:

```
*0\r\n
```

Whereas the encoding of an array consisting of the two bulk strings "hello" and "world" is:

```
*2\r\n$5\r\nhello\r\n$5\r\nworld\r\n
```

As you can see, after the *<count>CRLF part prefixing the array, the other data types that compose the array are concatenated one after the other. For example, an Array of three integers is encoded as follows:

```
*3\r\n:1\r\n:2\r\n:3\r\n
```

Arrays can contain mixed data types. For instance, the following encoding is of a list of four integers and a bulk string:

```
*5\r\n:1\r\n:2\r\n
```

```
:3\r\n
:4\r\n
$5\r\n
hello\r\n
```

(The raw RESP encoding is split into multiple lines for readability).

The first line the server sent is `*5\r\n`. This numeric value tells the client that five reply types are about to follow it. Then, every successive reply constitutes an element in the array.

All of the aggregate RESP types support nesting. For example, a nested array of two arrays is encoded as follows:

```
*2\r\n
*3\r\n
:1\r\n
:2\r\n
:3\r\n
*2\r\n
+Hello\r\n
-World\r\n
```

(The raw RESP encoding is split into multiple lines for readability).

The above encodes a two-element array. The first element is an array that, in turn, contains three integers (1, 2, 3). The second element is another array containing a simple string and an error.



Multi bulk reply:

In some places, the RESP Array type may be referred to as *multi bulk*. The two are the same.

Null arrays

Whereas RESP3 has a dedicated data type for null values, RESP2 has no such type. Instead, due to historical reasons, the representation of null values in RESP2 is via predetermined forms of the Bulk Strings and arrays types.

Null arrays exist as an alternative way of representing a null value. For instance, when the BLPOP command times out, it returns a null array.

The encoding of a null array is that of an array with the length of -1, i.e.:

```
*-1\r\n
```

When Redis replies with a null array, the client should return a null object rather than an empty array. This is necessary to distinguish between an empty list and a different condition (for instance, the timeout condition of the BLPOP command).

Null elements in arrays

Single elements of an array may be null bulk string. This is used in Redis replies to signal that these elements are missing and not empty strings. This can happen, for example, with the SORT command when used with the GET pattern option if the specified key is missing.

Here's an example of an array reply containing a null element:

```
*3\r\n
$5\r\n
hello\r\n
$-1\r\n
$5\r\n
world\r\n
```

Above, the second element is null. The client library should return to its caller something like this:

```
["hello",nil,"world"]
```

Nulls

The null data type represents non-existent values.

Nulls' encoding is the underscore (_) character, followed by the CRLF terminator (\r\n). Here's Null's raw RESP encoding:

```
_ \r\n
```



Null Bulk String, Null Arrays and Nulls:

Due to historical reasons, RESP2 features two specially crafted values for representing null values of bulk strings and arrays. This duality has always been a redundancy that added zero semantical value to the protocol itself.

The null type, introduced in RESP3, aims to fix this wrong.

Booleans

RESP booleans are encoded as follows:

```
#<t|f>\r\n
```

- The octothorpe character (#) as the first byte.
- A t character for true values, or an f character for false ones.
- The CRLF terminator.

Doubles

The Double RESP type encodes a double-precision floating point value. Doubles are encoded as follows:

```
, [<+|->] <integral> [ . <fractional> ] [ <E|e> [ sign ] <exponent> ] \r\n
```

- The comma character (,) as the first byte.
- An optional plus (+) or minus (-) as the sign.
- One or more decimal digits (0..9) as an unsigned, base-10 integral value.
- An optional dot (.), followed by one or more decimal digits (0..9) as an unsigned, base-10 fractional value.

- An optional capital or lowercase letter E (E or e), followed by an optional plus (+) or minus (-) as the exponent's sign, ending with one or more decimal digits (0..9) as an unsigned, base-10 exponent value.
- The CRLF terminator.

Here's the encoding of the number 1.23:

```
,1.23\r\n
```

Because the fractional part is optional, the integer value of ten (10) can, therefore, be RESP-encoded both as an integer as well as a double:

```
:10\r\n
,10\r\n
```

In such cases, the Redis client should return native integer and double values, respectively, providing that these types are supported by the language of its implementation.

The positive infinity, negative infinity and NaN values are encoded as follows:

```
,inf\r\n
,-inf\r\n
,nan\r\n
```

Big numbers

This type can encode integer values outside the range of signed 64-bit integers.

Big numbers use the following encoding:

```
([+|-]<number>\r\n
```

- The left parenthesis character (() as the first byte.
- An optional plus (+) or minus (-) as the sign.
- One or more decimal digits (0..9) as an unsigned, base-10 value.
- The CRLF terminator.

Example:

```
(3492890328409238509324850943850943825024385\r\n
```

Big numbers can be positive or negative but can't include fractionals. Client libraries written in languages with a big number type should return a big number. When big numbers aren't supported, the client should return a string and, when possible, signal to the caller that the reply is a big integer (depending on the API used by the client library).

Bulk errors

This type combines the purpose of simple errors with the expressive power of bulk strings.

It is encoded as:

```
!<length>\r\n<error>\r\n
```

- An exclamation mark (!) as the first byte.
- One or more decimal digits (0..9) as the error's length, in bytes, as an unsigned, base-10 value.
- The CRLF terminator.
- The error itself.
- A final CRLF.

As a convention, the error begins with an uppercase (space-delimited) word that conveys the error message.

For instance, the error "SYNTAX invalid syntax" is represented by the following protocol encoding:

```
!21\r\n  
SYNTAX invalid syntax\r\n
```

(The raw RESP encoding is split into multiple lines for readability).

Verbatim strings

This type is similar to the bulk string, with the addition of providing a hint about the data's encoding.

A verbatim string's RESP encoding is as follows:

```
=<length>\r\n<encoding>:<data>\r\n
```

- An equal sign (=) as the first byte.
- One or more decimal digits (0..9) as the string's total length, in bytes, as an unsigned, base-10 value.
- The CRLF terminator.
- Exactly three (3) bytes represent the data's encoding.
- The colon (:) character separates the encoding and data.
- The data.
- A final CRLF.

Example:

```
=15\r\n  
txt:Some string\r\n
```

(The raw RESP encoding is split into multiple lines for readability).

Some client libraries may ignore the difference between this type and the string type and return a native string in both cases. However, interactive clients, such as command line interfaces (e.g., redis-cli), can use this type and know that their output should be presented to the human user as is and without quoting the string.

For example, the Redis command INFO outputs a report that includes newlines. When using RESP3, redis-cli displays it correctly because it is sent as a Verbatim String reply (with its three bytes being "txt"). When using RESP2, however, the redis-cli is hard-coded to look for the INFO command to ensure its correct display to the user.

Maps

The RESP map encodes a collection of key-value tuples, i.e., a dictionary or a hash.

It is encoded as follows:

```
%<number-of-entries>\r\n<key-1><value-1>...<key-n><value-n>
```

- A percent character (%) as the first byte.
- One or more decimal digits (0..9) as the number of entries, or key-value tuples, in the map as an unsigned, base-10 value.
- The CRLF terminator.
- Two additional RESP types for every key and value in the map.

For example, the following JSON object:

```
{  
  "first": 1,  
  "second": 2  
}
```

Can be encoded in RESP like so:

```
%2\r\n  
+first\r\n  
:1\r\n  
+second\r\n  
:2\r\n
```

(The raw RESP encoding is split into multiple lines for readability).

Both map keys and values can be any of RESP's types.

Redis clients should return the idiomatic dictionary type that their language provides.

However, low-level programming languages (such as C, for example) will likely return an array along with type information that indicates to the caller that it is a dictionary.



Map pattern in RESP2:

RESP2 doesn't have a map type. A map in RESP2 is represented by a flat array containing the keys and the values. The first element is a key, followed by the corresponding value, then the next key and so on, like this: `key1, value1, key2, value2, ....`

Attributes

The attribute type is exactly like the Map type, but instead of a % character as the first byte, the | character is used. Attributes describe a dictionary exactly like the Map type. However the client should not consider such a dictionary part of the reply, but as auxiliary data that augments the reply.

Note: in the examples below, indentation is shown only for clarity; the additional whitespace would not be part of a real reply.

For example, newer versions of Redis may include the ability to report the popularity of keys for every executed command. The reply to the command MGET a b may be the following:

```
|1\r\n  +key-popularity\r\n  %2\r\n    $1\r\n    a\r\n    ,0.1923\r\n    $1\r\n    b\r\n    ,0.0012\r\n*2\r\n  :2039123\r\n  :9543892\r\n
```

The actual reply to MGET is just the two item array [2039123, 9543892]. The returned attributes specify the popularity, or frequency of requests, given as floating point numbers ranging from 0.0 to 1.0, of the keys mentioned in the original command. Note: the actual implementation in Redis may differ.

When a client reads a reply and encounters an attribute type, it should read the attribute, and continue reading the reply. The attribute reply should be accumulated separately, and the user should have a way to access such attributes. For instance, if we imagine a session in an higher level language, something like this could happen:

```
> r = Redis.new
#<Redis client>
> r.mget("a", "b")
#<Redis reply>
> r
[2039123, 9543892]
```

```
> r.attrs
{:key-popularity => {:a => 0.1923, :b => 0.0012}}
```

Attributes can appear anywhere before a valid part of the protocol identifying a given type, and supply information only about the part of the reply that immediately follows. For example:

```
*3\r\n
:1\r\n
:2\r\n
|1\r\n
+ttl\r\n
:3600\r\n
:3\r\n
```

In the above example the third element of the array has associated auxiliary information of `{ttl:3600}`. Note that it's not up to the client library to interpret the attributes, but it should pass them to the caller in a sensible way.

Sets

Sets are somewhat like [Arrays](#) but are unordered and should only contain unique elements.

RESP set's encoding is:

```
~<number-of-elements>\r\n<element-1>...<element-n>
```

- A tilde (~) as the first byte.
- One or more decimal digits (0..9) as the number of elements in the set as an unsigned, base-10 value.
- The CRLF terminator.
- An additional RESP type for every element of the Set.

Clients should return the native set type if it is available in their programming language. Alternatively, in the absence of a native set type, an array coupled with type information can be used (in C, for example).

Pushes

RESP's pushes contain out-of-band data. They are an exception to the protocol's request-response model and provide a generic *push mode* for connections.

Push events are encoded similarly to arrays, differing only in their first byte:

```
><number-of-elements>\r\n<element-1>...<element-n>
```

- A greater-than sign (>) as the first byte.
- One or more decimal digits (0..9) as the number of elements in the message as an unsigned, base-10 value.
- The CRLF terminator.
- An additional RESP type for every element of the push event.

Pushed data may precede or follow any of RESP's data types but never inside them. That means a client won't find push data in the middle of a map reply, for example. It also means that pushed data may appear before or after a command's reply, as well as by itself (without calling any command).

Clients should react to pushes by invoking a callback that implements their handling of the pushed data.

Client handshake

New RESP connections should begin the session by calling the HELLO command. This practice accomplishes two things:

1. It allows servers to be backward compatible with RESP2 versions. This is needed in Redis to make the transition to version 3 of the protocol gentler.
2. The HELLO command returns information about the server and the protocol that the client can use for different goals.

The HELLO command has the following high-level syntax:

```
HELLO <protocol-version> [optional-arguments]
```

The first argument of the command is the protocol version we want the connection to be set. By default, the connection starts in RESP2 mode. If we specify a connection version that is

too big and unsupported by the server, it should reply with a `-NOPROTO` error. Example:

```
Client: HELLO 4
Server: -NOPROTO sorry, this protocol version is not supported.
```

At that point, the client may retry with a lower protocol version.

Similarly, the client can easily detect a server that is only able to speak RESP2:

```
Client: HELLO 3
Server: -ERR unknown command 'HELLO'
```

The client can then proceed and use RESP2 to communicate with the server.

Note that even if the protocol's version is supported, the `HELLO` command may return an error, perform no action and remain in RESP2 mode. For example, when used with invalid authentication credentials in the command's optional AUTH clause:

```
Client: HELLO 3 AUTH default mypassword
Server: -ERR invalid password
(the connection remains in RESP2 mode)
```

A successful reply to the `HELLO` command is a map reply. The information in the reply is partly server-dependent, but certain fields are mandatory for all the RESP3 implementations:

- **server:** "redis" (or other software name).
- **version:** the server's version.
- **proto:** the highest supported version of the RESP protocol.

In Redis' RESP3 implementation, the following fields are also emitted:

- **id:** the connection's identifier (ID).
- **mode:** "standalone", "sentinel" or "cluster".
- **role:** "master" or "replica".
- **modules:** list of loaded modules as an Array of Bulk Strings.

Sending commands to a Redis server

Now that you are familiar with the RESP serialization format, you can use it to help write a Redis client library. We can further specify how the interaction between the client and the server works:

- A client sends the Redis server an array consisting of only bulk strings.
- A Redis server replies to clients, sending any valid RESP data type as a reply.

So, for example, a typical interaction could be the following.

The client sends the command `LLEN mylist` to get the length of the list stored at the key *mylist*. Then the server replies with an integer reply as in the following example (C: is the client, S: the server).

```
C: *2\r\n
C: $4\r\n
C: LLEN\r\n
C: $6\r\n
C: mylist\r\n

S: :48293\r\n
```

As usual, we separate different parts of the protocol with newlines for simplicity, but the actual interaction is the client sending `*2\r\n$4\r\nLLEN\r\n$6\r\nmylist\r\n` as a whole.

Multiple commands and pipelining

A client can use the same connection to issue multiple commands. Pipelining is supported, so multiple commands can be sent with a single write operation by the client. The client can skip reading replies and continue to send the commands one after the other. All the replies can be read at the end.

For more information, see [Pipelining](#).

Inline commands

Sometimes you may need to send a command to the Redis server but only have `telnet` available. While the Redis protocol is simple to implement, it is not ideal for interactive

sessions, and `redis-cli` may not always be available. For this reason, Redis also accepts commands in the *inline command* format.

The following example demonstrates a server/client exchange using an inline command (the server chat starts with S:, the client chat with C:):

```
C: PING
S: +PONG
```

Here's another example of an inline command where the server returns an integer:

```
C: EXISTS somekey
S: :0
```

Basically, to issue an inline command, you write space-separated arguments in a telnet session. Since no command starts with `*` (the identifying byte of RESP Arrays), Redis detects this condition and parses your command inline.

High-performance parser for the Redis protocol

While the Redis protocol is human-readable and easy to implement, its implementation can exhibit performance similar to that of a binary protocol.

RESP uses prefixed lengths to transfer bulk data. That makes scanning the payload for special characters unnecessary (unlike parsing JSON, for example). For the same reason, quoting and escaping the payload isn't needed.

Reading the length of aggregate types (for example, bulk strings or arrays) can be processed with code that performs a single operation per character while at the same time scanning for the CR character.

Example (in C):

```
#include <stdio.h>

int main(void) {
    unsigned char *p = "$123\r\n";
    int len = 0;
```

```

p++;
while(*p != '\r') {
    len = (len*10)+(*p - '0');
    p++;
}

/* Now p points at '\r', and the length is in len. */
printf("%d\n", len);
return 0;
}

```

After the first CR is identified, it can be skipped along with the following LF without further processing. Then, the bulk data can be read with a single read operation that doesn't inspect the payload in any way. Finally, the remaining CR and LF characters are discarded without additional processing.

While comparable in performance to a binary protocol, the Redis protocol is significantly more straightforward to implement in most high-level languages, reducing the number of bugs in client software.

Tips for Redis client authors

- For testing purposes, use [Lua's type conversions](#) to have Redis reply with any RESP2/RESP3 needed. As an example, a RESP3 double can be generated like so:

```
EVAL "return { double = tonumber(ARGV[1]) }" 0 1e0
```

Redis



USE CASES

Vector database

Feature stores

Semantic cache

Caching

NoSQL database

Leaderboards

Data deduplication

Messaging

Authentication token storage

Fast-data ingest

Query caching

All solutions

INDUSTRIES

Financial Services

Gaming

Healthcare

Retail

All industries

COMPARE

Redis vs Elasticache

Redis vs Memcached

Redis vs Memory Store

Redis vs Source Available

COMPANY

Mission & values

Leadership

Careers

News

CONNECT

[Community](#)

[Events & webinars](#)

[News](#)

PARTNERS

[Amazon Web Services](#)

[Google Cloud](#)

[Microsoft Azure](#)

[All partners](#)

SUPPORT

[Professional services](#)

[Support](#)

[Trust](#)

[Privacy](#)

[Terms of use](#)

[Legal notices](#)